

08_tarea_grafico_barras_lineas_superstore

April 27, 2026

1 Tarea paso a paso: Gráfico Combinado Barras y Líneas

¡ATENCIÓN! Lee esto antes de empezar: Esta tarea está diseñada paso a paso para que afiances los conceptos vistos en clase sobre los **Bar + Line Charts**. Su objetivo es que practiques cómo combinar un gráfico de barras y una línea en el mismo gráfico, usando un eje Y secundario cuando las escalas de las variables son muy distintas.

Nota importante sobre la Inteligencia Artificial: Es tentador copiar y pegar el enunciado de estos ejercicios en ChatGPT o Claude para que te den la solución... **Hacer eso perjudicará gravemente tu aprendizaje. (CODEA UN POCO QUE TE ENTERES QUE HACES).** La única forma de asimilar la sintaxis de las herramientas de visualización es escribiéndolas. **Usa tu cerebro, actúa paso a paso, equivócate, lee la documentación de Seaborn y Plotly, busca tu fallo y ¡aprende de él!**

1.1 Preparación y Datos

Para esta tarea vamos a utilizar el dataset “**Superstore Sales**”, un dataset clásico de Business Intelligence que recoge las ventas de una tienda americana de 2014 a 2017. Cada fila representa un pedido con su categoría de producto, ventas totales, beneficio y región.

Es un dataset ideal para un **Bar + Line Chart** porque permite combinar dos magnitudes relacionadas pero con escalas muy distintas: - **Sales**: ingresos totales por ventas (en dólares, orden de magnitud: miles a millones). - **Profit**: beneficio neto obtenido, que puede ser negativo si hay pérdidas (mucho menor en escala que Sales).

Las barras representarán el volumen de ventas (¿cuánto vendemos?) y la línea el beneficio (¿cuánto ganamos?), que es el gráfico más usado en informes de negocio.

1. Descarga el dataset de Kaggle: [Superstore Sales Dataset](#)
2. Guarda el archivo como `superstore.csv` en la carpeta `data/` de tu proyecto.
3. En la siguiente celda, importa las librerías necesarias y carga el CSV en un DataFrame llamado `df`.

```
[1]: import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
import plotly.graph_objects as go
from plotly.subplots import make_subplots
```

```
# CARGA DE DATOS
df = pd.read_csv("data/Sample - Superstore.csv", encoding='latin-1')

df.head(5)
```

```
[1]:
```

	Row ID	Order ID	Order Date	Ship Date	Ship Mode	Customer ID	\
0	1	CA-2016-152156	11/8/2016	11/11/2016	Second Class	CG-12520	
1	2	CA-2016-152156	11/8/2016	11/11/2016	Second Class	CG-12520	
2	3	CA-2016-138688	6/12/2016	6/16/2016	Second Class	DV-13045	
3	4	US-2015-108966	10/11/2015	10/18/2015	Standard Class	SO-20335	
4	5	US-2015-108966	10/11/2015	10/18/2015	Standard Class	SO-20335	

	Customer Name	Segment	Country	City	...	\
0	Claire Gute	Consumer	United States	Henderson	...	
1	Claire Gute	Consumer	United States	Henderson	...	
2	Darrin Van Huff	Corporate	United States	Los Angeles	...	
3	Sean O'Donnell	Consumer	United States	Fort Lauderdale	...	
4	Sean O'Donnell	Consumer	United States	Fort Lauderdale	...	

	Postal Code	Region	Product ID	Category	Sub-Category	\
0	42420	South	FUR-BO-10001798	Furniture	Bookcases	
1	42420	South	FUR-CH-10000454	Furniture	Chairs	
2	90036	West	OFF-LA-10000240	Office Supplies	Labels	
3	33311	South	FUR-TA-10000577	Furniture	Tables	
4	33311	South	OFF-ST-10000760	Office Supplies	Storage	

	Product Name	Sales	Quantity	\
0	Bush Somerset Collection Bookcase	261.9600	2	
1	Hon Deluxe Fabric Upholstered Stacking Chairs,...	731.9400	3	
2	Self-Adhesive Address Labels for Typewriters b...	14.6200	2	
3	Bretford CR4500 Series Slim Rectangular Table	957.5775	5	
4	Eldon Fold 'N Roll Cart System	22.3680	2	

	Discount	Profit
0	0.00	41.9136
1	0.00	219.5820
2	0.00	6.8714
3	0.45	-383.0310
4	0.20	2.5164

[5 rows x 21 columns]

```
[2]: # Ver la información del dataset
df.describe(include="all")
```

```
[2]:
```

	Row ID	Order ID	Order Date	Ship Date	Ship Mode	\
count	9994.000000	9994	9994	9994	9994	

unique	NaN	5009	1237	1334	4
top	NaN	CA-2017-100111	9/5/2016	12/16/2015	Standard Class
freq	NaN	14	38	35	5968
mean	4997.500000	NaN	NaN	NaN	NaN
std	2885.163629	NaN	NaN	NaN	NaN
min	1.000000	NaN	NaN	NaN	NaN
25%	2499.250000	NaN	NaN	NaN	NaN
50%	4997.500000	NaN	NaN	NaN	NaN
75%	7495.750000	NaN	NaN	NaN	NaN
max	9994.000000	NaN	NaN	NaN	NaN

	Customer ID	Customer Name	Segment	Country	City \
count	9994	9994	9994	9994	9994
unique	793	793	3	1	531
top	WB-21850	William Brown	Consumer	United States	New York City
freq	37	37	5191	9994	915
mean	NaN	NaN	NaN	NaN	NaN
std	NaN	NaN	NaN	NaN	NaN
min	NaN	NaN	NaN	NaN	NaN
25%	NaN	NaN	NaN	NaN	NaN
50%	NaN	NaN	NaN	NaN	NaN
75%	NaN	NaN	NaN	NaN	NaN
max	NaN	NaN	NaN	NaN	NaN

	...	Postal Code	Region	Product ID	Category \
count	...	9994.000000	9994	9994	9994
unique	...	NaN	4	1862	3
top	...	NaN	West	OFF-PA-10001970	Office Supplies
freq	...	NaN	3203	19	6026
mean	...	55190.379428	NaN	NaN	NaN
std	...	32063.693350	NaN	NaN	NaN
min	...	1040.000000	NaN	NaN	NaN
25%	...	23223.000000	NaN	NaN	NaN
50%	...	56430.500000	NaN	NaN	NaN
75%	...	90008.000000	NaN	NaN	NaN
max	...	99301.000000	NaN	NaN	NaN

	Sub-Category	Product Name	Sales	Quantity	Discount \
count	9994	9994	9994.000000	9994.000000	9994.000000
unique	17	1850	NaN	NaN	NaN
top	Binders	Staple envelope	NaN	NaN	NaN
freq	1523	48	NaN	NaN	NaN
mean	NaN	NaN	229.858001	3.789574	0.156203
std	NaN	NaN	623.245101	2.225110	0.206452
min	NaN	NaN	0.444000	1.000000	0.000000
25%	NaN	NaN	17.280000	2.000000	0.000000
50%	NaN	NaN	54.490000	3.000000	0.200000

75%	NaN	NaN	209.940000	5.000000	0.200000
max	NaN	NaN	22638.480000	14.000000	0.800000

	Profit
count	9994.000000
unique	NaN
top	NaN
freq	NaN
mean	28.656896
std	234.260108
min	-6599.978000
25%	1.728750
50%	8.666500
75%	29.364000
max	8399.976000

[11 rows x 21 columns]

```
[3]: # Limpieza de nulos
df = df.dropna()
df
```

```
[3]:      Row ID      Order ID  Order Date  Ship Date  Ship Mode \
0         1  CA-2016-152156   11/8/2016  11/11/2016   Second Class
1         2  CA-2016-152156   11/8/2016  11/11/2016   Second Class
2         3  CA-2016-138688   6/12/2016   6/16/2016   Second Class
3         4  US-2015-108966  10/11/2015  10/18/2015   Standard Class
4         5  US-2015-108966  10/11/2015  10/18/2015   Standard Class
...
9989    9990  CA-2014-110422   1/21/2014   1/23/2014   Second Class
9990    9991  CA-2017-121258   2/26/2017   3/3/2017   Standard Class
9991    9992  CA-2017-121258   2/26/2017   3/3/2017   Standard Class
9992    9993  CA-2017-121258   2/26/2017   3/3/2017   Standard Class
9993    9994  CA-2017-119914   5/4/2017   5/9/2017   Second Class
```

	Customer ID	Customer Name	Segment	Country	City \
0	CG-12520	Claire Gute	Consumer	United States	Henderson
1	CG-12520	Claire Gute	Consumer	United States	Henderson
2	DV-13045	Darrin Van Huff	Corporate	United States	Los Angeles
3	SO-20335	Sean O'Donnell	Consumer	United States	Fort Lauderdale
4	SO-20335	Sean O'Donnell	Consumer	United States	Fort Lauderdale
...	...	...	...	...	...
9989	TB-21400	Tom Boeckenhauer	Consumer	United States	Miami
9990	DB-13060	Dave Brooks	Consumer	United States	Costa Mesa
9991	DB-13060	Dave Brooks	Consumer	United States	Costa Mesa
9992	DB-13060	Dave Brooks	Consumer	United States	Costa Mesa
9993	CC-12220	Chris Cortes	Consumer	United States	Westminster

	...	Postal Code	Region	Product ID	Category	Sub-Category	\
0	...	42420	South	FUR-BO-10001798	Furniture	Bookcases	
1	...	42420	South	FUR-CH-10000454	Furniture	Chairs	
2	...	90036	West	OFF-LA-10000240	Office Supplies	Labels	
3	...	33311	South	FUR-TA-10000577	Furniture	Tables	
4	...	33311	South	OFF-ST-10000760	Office Supplies	Storage	
...	...	...	...	...	...	...	
9989	...	33180	South	FUR-FU-10001889	Furniture	Furnishings	
9990	...	92627	West	FUR-FU-10000747	Furniture	Furnishings	
9991	...	92627	West	TEC-PH-10003645	Technology	Phones	
9992	...	92627	West	OFF-PA-10004041	Office Supplies	Paper	
9993	...	92683	West	OFF-AP-10002684	Office Supplies	Appliances	

		Product Name	Sales	Quantity	\
0		Bush Somerset Collection Bookcase	261.9600	2	
1	Hon Deluxe Fabric Upholstered Stacking Chairs,...		731.9400	3	
2	Self-Adhesive Address Labels for Typewriters b...		14.6200	2	
3	Bretford CR4500 Series Slim Rectangular Table		957.5775	5	
4	Eldon Fold 'N Roll Cart System		22.3680	2	
...	...	...	...	...	
9989		Ultra Door Pull Handle	25.2480	3	
9990	Tenex B1-RE Series Chair Mats for Low Pile Car...		91.9600	2	
9991		Aastra 57i VoIP phone	258.5760	2	
9992	It's Hot Message Books with Stickers, 2 3/4" x 5"		29.6000	4	
9993	Acco 7-Outlet Masterpiece Power Center, Wihtou...		243.1600	2	

	Discount	Profit
0	0.00	41.9136
1	0.00	219.5820
2	0.00	6.8714
3	0.45	-383.0310
4	0.20	2.5164
...	...	...
9989	0.20	4.1028
9990	0.00	15.6332
9991	0.20	19.3932
9992	0.00	13.3200
9993	0.00	72.9480

[9994 rows x 21 columns]

1.2 Paso 1: Preparar los Datos

Objetivo: Extraer el año de la columna de fechas y agregar las ventas y el beneficio por año.

Instrucciones: 1. Convierte la columna 'Order Date' a tipo fecha con `pd.to_datetime(df['Order Date'])`. 2. Crea una nueva columna 'Year' extrayendo el

año: `df['Year'] = df['Order Date'].dt.year`. 3. Agrupa el DataFrame por 'Year' y calcula:
- La suma de Sales. - La suma de Profit.

Usa `.agg({'Sales': 'sum', 'Profit': 'sum'})` y guarda en `df_agrupado`. Aplica `.reset_index()` al final. 4. Muestra `df_agrupado`. Deberías ver 4 filas (2014, 2015, 2016, 2017).

```
[4]: # Order Date a fecha y saco el año
# Uso sum en ventas y beneficio (los dos se acumulan, no se promedian)
df['Order Date'] = pd.to_datetime(df['Order Date'])
df['Year'] = df['Order Date'].dt.year

df_agrupado = df.groupby('Year').agg({
    'Sales': 'sum',
    'Profit': 'sum'
}).reset_index()
df_agrupado
```

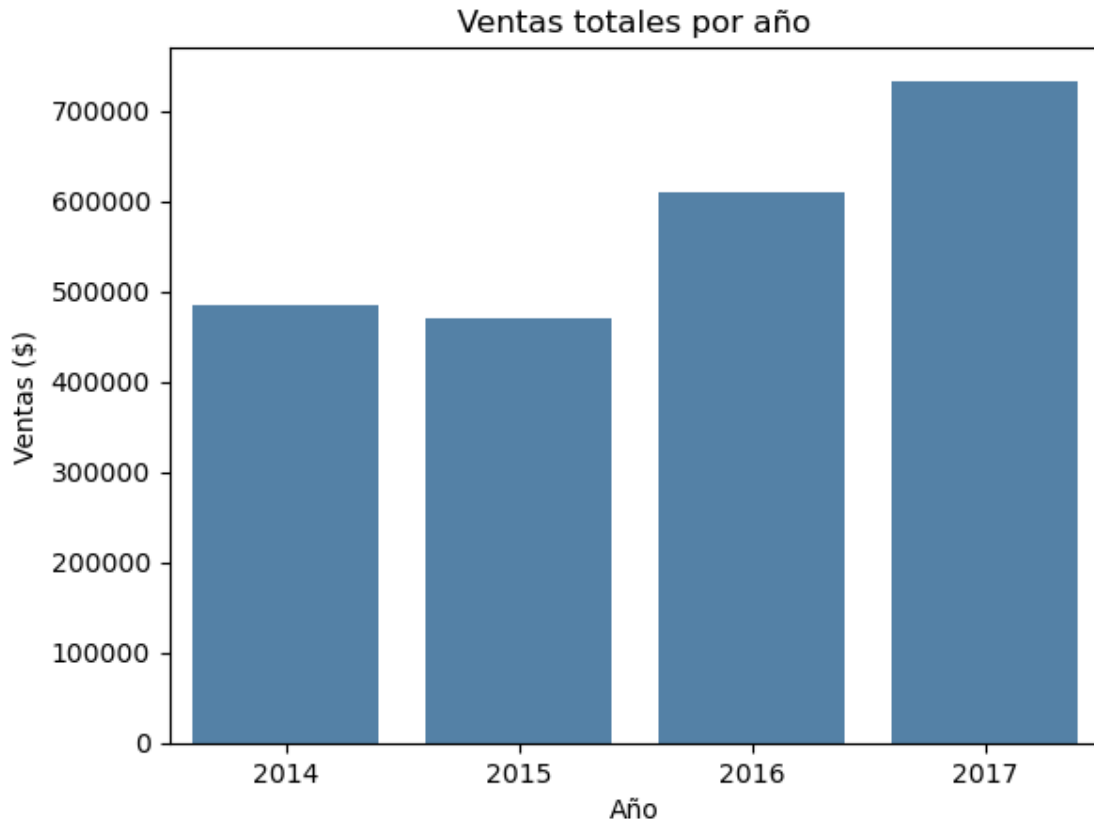
```
[4]:   Year      Sales      Profit
0  2014  484247.4981  49543.9741
1  2015  470532.5090  61618.6037
2  2016  609205.5980  81795.1743
3  2017  733215.2552  93439.2696
```

1.3 Paso 2: Gráfico de Barras Simple (Solo Ventas)

Objetivo: Visualizar únicamente las ventas por año con un gráfico de barras de Seaborn antes de añadir la línea de beneficio.

Instrucciones: 1. Usa `sns.barplot()` con `df_agrupado`. 2. Asigna `x='Year'` e `y='Sales'`. Pon un color neutro (por ejemplo `color='steelblue'`). 3. Añade título y etiquetas de ejes con `plt.title(...)`, `plt.xlabel(...)`, `plt.ylabel(...)`. 4. Muestra el gráfico. ¿Se aprecia la tendencia de crecimiento en ventas?

```
[5]: # grafico de barras
sns.barplot(
    x='Year',
    data=df_agrupado,
    y='Sales',
    color='steelblue'           # color uniforme: aquí no hay categoría que
    ↪diferenciar
)
plt.title('Ventas totales por año')
plt.xlabel('Año')
plt.ylabel('Ventas ($)')
plt.show()
```



1.4 Paso 3: Gráfico Combinado con Seaborn + Matplotlib (twinx)

Objetivo: Superponer la línea de beneficio sobre las barras de ventas usando un eje Y secundario, de modo que ambas métricas sean legibles a la vez.

Instrucciones: 1. Crea la figura con `fig, ax1 = plt.subplots(figsize=(8, 4))`. 2. Dibuja las barras en `ax1` con `sns.barplot(data=df_agrupado, x='Year', y='Sales', color='steelblue', alpha=0.8, ax=ax1)`. Añade `ax1.set_axisbelow(True)`. 3. Crea el segundo eje con `ax2 = ax1.twinx()`. 4. Dibuja la línea de beneficio en `ax2` con `sns.lineplot(data=df_agrupado, x='Year', y='Profit', ax=ax2, color='darkorange', marker='D', linewidth=2)`. 5. Etiqueta el eje derecho con `ax2.set_ylabel('Beneficio ($)', color='darkorange')` y aplica `ax2.tick_params(axis='y', labelcolor='darkorange')` para que los números del eje también sean naranjas. 6. Desactiva la cuadrícula del eje secundario con `ax2.grid(visible=False)` para que no se solape con la del primario. 7. Añade un título y muestra el gráfico.

```
[6]: # Combinado: barras de Sales + línea de Profit en eje Y secundario
fig, ax1 = plt.subplots(figsize=(8, 4))

sns.barplot(
    data=df_agrupado,
```

```

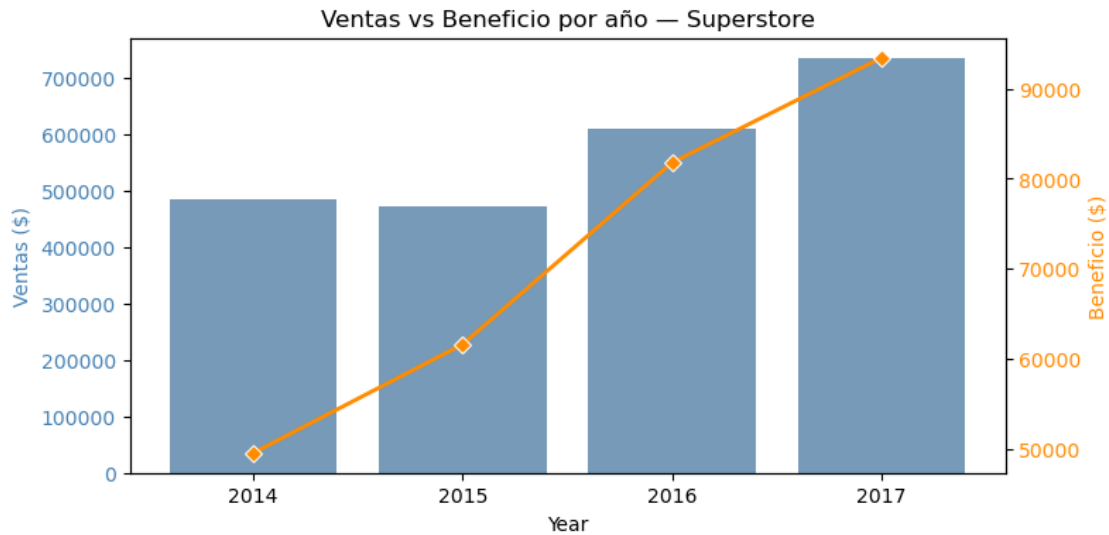
x='Year',
y='Sales',
color='steelblue',
alpha=0.8,          # transparencia para que se vea la línea encima
ax=ax1
)
ax1.set_axisbelow(True)
ax1.set_ylabel('Ventas ($)', color='steelblue')
ax1.tick_params(axis='y', labelcolor='steelblue')

ax2 = ax1.twinx()

# Paso la Serie sola para que use el índice posicional 0..3 igual que las
↳ barras.
# Si paso x='Year' se va fuera del area de barras (barplot usa ticks categóricos
# y lineplot ticks numéricos).
line_color = 'darkorange'
ax2.set_ylabel('Beneficio ($)', color=line_color)
sns.lineplot(
    data=df_agrupado['Profit'],
    alpha=1,
    marker='D',          # diamante para resaltar cada punto
    ax=ax2,
    linewidth=2,
    color=line_color
)
ax2.tick_params(axis='y', labelcolor=line_color)
ax2.set_axisbelow(True)
ax2.grid(visible=False)    # quito la rejilla del 2º eje para no duplicar
↳ la del primero

plt.title('Ventas vs Beneficio por año - Superstore')
plt.show()

```

1.5 Paso 4: Gráfico Combinado con Plotly por Categoría

Objetivo: Reproducir el gráfico con Plotly usando `make_subplots` con eje Y secundario, y añadir el desglose por categoría de producto en las barras para un análisis más detallado.

Instrucciones: 1. Crea un nuevo DataFrame `df_categoria` agrupando por `['Year', 'Category']` y calculando la suma de `Sales` y `Profit`. Aplica `.reset_index()`. 2. Crea también `df_profit_total` agrupando solo por `'Year'` y calculando la suma de `Profit`. Aplica `.reset_index()`. 3. Crea la figura con `fig = make_subplots(specs=[[{"secondary_y": True}]])`. 4. Con un bucle `for category in df_categoria['Category'].unique()`, filtra `df_categoria` por cada categoría y añade un `go.Bar(...)` a la figura con `secondary_y=False`. Activa `barmode='group'` en el layout. 5. Añade la línea de beneficio total con `go.Scatter(x=..., y=..., mode='markers+lines', name='Beneficio Total')` con `secondary_y=True`. 6. Configura el layout con títulos para ambos ejes Y y un título general. 7. Muestra el gráfico con `fig.show()`.

```
[7]: # Hacemos lo mismo pero por año y categoría (alimenta las barras agrupadas)
df_categoria = df.groupby(['Year', 'Category']).agg({
    'Sales': 'sum',
    'Profit': 'sum'
}).reset_index()

# Datos para la línea (Profit total por año, sin desglose)
df_profit_total = df.groupby('Year').agg({
    'Profit': 'sum'
}).reset_index()

# Crear un subplot con especificación para un segundo eje Y
fig = make_subplots(specs=[[{"secondary_y": True}]])
```

```

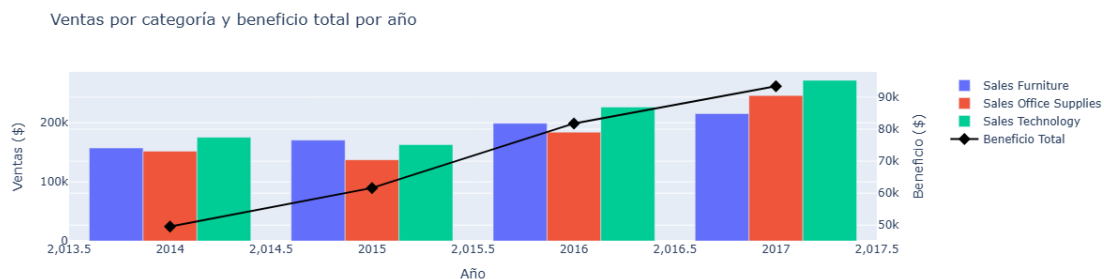
# Añadir el gráfico de barras para 'Sales' diferenciando por 'Category'
for category in df_categoria['Category'].unique():
    fig.add_trace(
        go.Bar(
            x=df_categoria[df_categoria['Category'] == category]['Year'],
            y=df_categoria[df_categoria['Category'] == category]['Sales'],
            name=f'Sales {category}',
            offsetgroup=category,
        ),
        secondary_y=False,
    )

# Añadir el gráfico de líneas para 'Profit' total
fig.add_trace(
    go.Scatter(
        x=df_profit_total['Year'],
        y=df_profit_total['Profit'],
        name='Beneficio Total',
        mode='markers+lines',
        line=dict(color='black', width=2), # negro para destacar sobre las
        ↪barras de colores
        marker=dict(symbol='diamond', size=10)
    ),
    secondary_y=True,
)

# Actualizar el layout del gráfico
fig.update_layout(
    title='Ventas por categoría y beneficio total por año',
    xaxis_title='Año',
    yaxis_title='Ventas ($)',
    yaxis2_title='Beneficio ($)',
    barmode='group', # barras agrupadas, no apiladas
)

fig.show()

```



1.5.1 Interpretación del Gráfico Combinado Barras y Líneas

Este gráfico final integra **dos niveles de análisis** en una sola figura:

1. **Las Barras (Ventas por Categoría):** permiten comparar, dentro de cada año, qué categoría de producto genera más ingresos. El eje izquierdo tiene la escala de ventas totales en dólares.
2. **La Línea (Beneficio Total):** muestra la evolución del beneficio neto en el eje derecho. Aunque las ventas crezcan, el beneficio puede no hacerlo proporcionalmente (por ejemplo, si aumentan los descuentos o los costes).
3. **Conclusión clave:** este gráfico es el estándar en cuadros de mando de negocio porque responde de un vistazo a la pregunta “¿Vendemos más y ganamos más?”, que no siempre tiene la misma respuesta.

1.6 Reflexión Final del Alumno

Tómate un rato para reflexionar. * El **gráfico combinado de barras y líneas** es uno de los más usados en entornos empresariales y de negocio porque permite comparar un volumen (barras) con una tasa o métrica relacionada (línea) en el mismo espacio visual. * Recuerda la regla de oro vista en clase: **no añadas más de dos líneas** al gráfico, o se volverá ilegible. Si necesitas mostrar más variables, considera dividirlo en varios subgráficos. * Como reto adicional: ¿podrías repetir el análisis agrupando por 'Region' en lugar de por 'Category'? ¿O cambiar el eje X a meses en lugar de años para ver la estacionalidad de las ventas?

Asegúrate de que cada celda tenga sus respectivos comentarios.

Por Región me quedan 4 barras, Central East South West, útil para pillar si alguna vende mucho pero pierde margen. Y por meses se vería la estacionalidad, pienso que noviembre y diciembre se disparan por navidad.

```
[8]: # Reto: mismo gráfico pero por Region en vez de Category
df_region = df.groupby(['Year', 'Region']).agg({
    'Sales': 'sum',
    'Profit': 'sum'
}).reset_index()

df_profit_total_r = df.groupby('Year').agg({
    'Profit': 'sum'
}).reset_index()

fig = make_subplots(specs=[[{"secondary_y": True}]])

# Una barra por cada región (Central, East, South, West)
for region in df_region['Region'].unique():
    fig.add_trace(
        go.Bar(
            x=df_region[df_region['Region'] == region]['Year'],
```

```

        y=df_region[df_region['Region'] == region]['Sales'],
        name=f'Sales {region}',
        offsetgroup=region,
    ),
    secondary_y=False,
)

fig.add_trace(
    go.Scatter(
        x=df_profit_total_r['Year'],
        y=df_profit_total_r['Profit'],
        name='Beneficio Total',
        mode='markers+lines',
        line=dict(color='black', width=2),
        marker=dict(symbol='diamond', size=10)
    ),
    secondary_y=True,
)

fig.update_layout(
    title='Ventas por Región y beneficio total por año',
    xaxis_title='Año',
    yaxis_title='Ventas ($)',
    yaxis2_title='Beneficio ($)',
    barmode='group',
)

fig.show()

```

